Aaron Jones

CS 499: Computer Science Capstone — Professional Self-Assessment

Completing the Computer Science program at Southern New Hampshire University, and this capstone in particular, has given me a clearer picture of what I can actually do rather than just what I've been taught. I haven't committed to a narrow specialization within computer science, and at this point in a genuinely competitive field — one being reshaped by AI as quickly as it is — I'm not trying to brand myself around one. What I want this portfolio to demonstrate is more fundamental than that: that I'm capable, that I can take on a task I haven't necessarily done before, and that I can see it through to something that actually works. The three artifacts in this portfolio, each enhanced in a different core area of computer science, are the clearest evidence I have of that.

Collaborating in a Team Environment

Most of my coursework has been independent, structured project work, so the clearest example I can point to of working within a shared, collaborative environment is the structured peer-review process built into courses like CS 465 (Full Stack Development), where I wrote discussion posts and peer responses to classmates' work — including feedback exchanged with classmates Jonathon Miller and Attila Bordan on their own project approaches. That format is asynchronous and lower-stakes than a shared codebase, and I want to be honest about that rather than overstate it, but it's still a real exercise in the skill underneath real teamwork: reading someone else's technical decisions, understanding the reasoning behind them, and giving feedback that's useful rather than just correct. It's the same skill a code review from a teammate or a manager would require, just in a classroom's version of that exchange.

Communicating with Stakeholders

Two of the artifacts in this portfolio were built directly around a specific audience's stated needs rather than an abstract specification. The Contact/Task/Appointment management services (Software Design and Engineering) were built for a named fictional client, Grand Strand Systems, which meant translating plain-language client requirements — a phone number that has to be exactly ten digits, an appointment date that can't be in the past — into concrete validation rules and the automated tests that enforce them. The Grazioso Salvare Rescue Animal Dashboard (Databases) was built around a real rescue organization's actual workflow: staff need to filter animals by rescue-type profile (water rescue, mountain rescue, disaster tracking) without knowing anything about the database underneath it, which meant the technical design had to disappear behind an interface a non-technical stakeholder could just use. Outside this portfolio, I got more direct practice translating technical results into plain language in MAT 350 (Applied Linear Algebra), where my discussion posts had to explain things like singular value decomposition and image compression in terms a reader without the same math background could actually follow, rather than just reporting the result.

Data Structures and Algorithms

The clearest demonstration of this in the portfolio is the Milestone Three enhancement to the ABCU Advising Program, where I replaced a vector-based linear search with a hash map keyed by course number — turning the program's most frequent operation, looking up a single course, from an O(n) scan into an O(1) average-case lookup, and documenting the real trade-off that comes with it (the hash map gives up the vector's inherent ordering, which has to be rebuilt only when a sorted listing is actually requested). Outside this portfolio, my strongest algorithms work was in CS 370 (Artificial Intelligence), where I built a reinforcement-learning agent — implementing Q-learning, A2C, and REINFORCE — that reached a 100% win rate on its task. That project required reasoning about an algorithm's behavior over time rather than just its output on a single input, which is a different kind of algorithmic thinking than anything in this portfolio and one I'd want a future employer to know I have.

Software Engineering and Database

This is the area where this portfolio's artifacts carry the most direct weight. The Milestone Two enhancement introduced a repository interface in front of what had been three services directly coupled to an in-memory HashMap, added a file-backed implementation so data actually persists, and added the integration tests, Maven build, and GitHub Actions CI workflow needed to support that — none of which existed in the original artifact. The Milestone Four enhancement completed a CRUD module that, despite its name, had never implemented update or delete, and added a compound index built around the specific fields the application's own queries actually use together. Outside the portfolio, I maintain a self-hosted home server running a Docker-based stack with its own backend scripting and API work, which is where most of my hands-on infrastructure and deployment experience actually comes from — it's informal compared to a classroom project, but it's sustained, real software I'm personally responsible for keeping running.

Security

The Milestone Four enhancement is this portfolio's direct evidence of a security mindset: removing hardcoded database credentials in favor of environment variables, fixing a connection check that silently accepted bad credentials because of how PyMongo's lazy connection behavior actually works, escaping credentials before they're placed in a connection URI, and adding guardrails so an empty query can't silently trigger a mass update or delete. These weren't hypothetical risks — each one was a real defect I found and fixed in a real artifact. Outside the portfolio, CS 370's AI ethics work, including a white paper on GDPR, pushed me to think about privacy and data protection from a policy and design angle rather than only a code-level one, which I think is a useful complement to the hands-on fixes in Milestone Four rather than a repeat of them.

How the Artifacts Fit Together

The three artifacts in this portfolio were chosen to cover three different, necessary layers of the same kind of work: the Contact/Task/Appointment services demonstrate sound object-oriented design and the discipline of testing against real requirements; the ABCU Advising Program demonstrates the ability to recognize and fix a real performance and design trade-off at the

data-structure level; and the Grazioso Salvare Dashboard demonstrates taking a full application — database, backend logic, and a usable front end — and making it both more complete and more secure. None of them were already finished, polished projects before this course; each one had a real, identifiable problem that I diagnosed and fixed, which is closer to what maintaining software in a job actually looks like than building something from a blank page once and never touching it again. Taken together, I think they make a straightforward case: given an existing system and a real problem in it, I can figure out what's actually wrong, fix it correctly, and prove that the fix works.

AI Usage Acknowledgment: Generative AI (Claude) was used to help organize this self-assessment around the required topics and draft language based on my own stated career direction, the real coursework and projects described above, and the enhancement work completed for this capstone. The examples chosen, the framing of my career goals, and the final wording are my own.